Faculté des Sciences Aïn Chock
Université Hassan II de Casablanca

Master Ingénierie des Systèmes d'Information (ISI)

Rapport du Projet

Application Web de Gestion des Rendez-vous

Module : DWFS

Réalisé par : Imane Ait Elmir / Assia Mouad

Encadré par : Pr. CHANYOUR

Année universitaire : 2025–2026

Table des matières

1 Présentation Générale du Projet **3**
1.1 Contexte du projet 3
1.2 Problématique 3
1.3 Objectifs du projet 4
1.4 Technologies utilisées 4
1.4.1 Outils de développement 5
1.4.2 Technologies Web 6
1.4.3 Outils de rédaction du rapport 7

2 Analyse et Conception de la Solution **8**
2.1 Analyse fonctionnelle 8
2.2 Architecture globale de l'application 8
2.3 Modélisation des données 9
2.4 Conception des API REST 10

3 Réalisation et évaluation **11**
3.1 Implémentation 11
3.1.1 Arborescence backend 12
3.1.2 Arborescence frontend-dashboard 13
3.2 Tests et validation 14
3.3 Résultats obtenus 17
3.4 Perspectives d'amélioration 17

4 Conclusion Générale **18**

Table des figures

1.1 Framework Angular pour le développement frontend 4
1.2 Node.js pour le développement du serveur backend 5
1.3 Framework Express.js pour la création des API REST 5
1.4 Base de données relationnelle SQLite 5
1.5 Visual Studio Code – Environnement de développement 5
1.6 Postman – Outil de test des API REST 6
1.7 HTML – Structure des pages web 6
1.8 CSS – Mise en forme de l'interface 6
1.9 JavaScript – Logique côté client 6
1.10 TypeScript – Développement Angular 7
1.11 LaTeX – Système de composition de documents 7

2.1 Architecture du projet 9

3.1 Structure du projet backend 12
3.2 Structure du projet frontend 13
3.3 Test à l'aide de Postman 14
3.4 Register 15
3.5 Login 15
3.6 Creation d'un rendez-vous 15
3.7 Calendrier des rendez-vous 16
3.8 Supprimer un rendez-vous 16
3.9 Add-appointment en cas d'ajout 16
3.10 Add-appointment en cas de mis a jour 17

Chapitre 1

Présentation Générale du Projet

1.1 Contexte du projet

Avec l'évolution rapide des technologies web, les applications orientées services et basées sur des architectures distribuées occupent une place centrale dans les systèmes d'information modernes. Dans ce contexte, la maîtrise des API REST, des frameworks backend tels que Node.js avec Express, ainsi que des frameworks frontend comme Angular, constitue une compétence essentielle pour tout étudiant en Master ISI.

Ce projet académique s'inscrit dans cette optique et vise à consolider les connaissances théoriques acquises en cours à travers la réalisation d'une application web complète, fonctionnelle et structurée, reposant sur une architecture client/serveur.

Le projet consiste à développer une application web de gestion des rendez-vous, intégrant :

- Une gestion des utilisateurs (inscription et connexion),
- Un tableau de bord interactif pour la planification, la modification et le suivi des rendez-vous à travers un calendrier.

1.2 Problématique

La gestion des rendez-vous constitue un élément essentiel dans de nombreux domaines tels que l'administration, les services, la formation ou la santé. Une gestion manuelle ou mal organisée peut engendrer plusieurs problèmes :

- conflits de dates et d'horaires,
- pertes ou oublis de rendez-vous,
- manque de visibilité globale sur le planning,
- difficulté de suivi et de mise à jour.

La problématique principale abordée par ce projet est donc la suivante : *Comment concevoir une application web fiable, sécurisée et évolutive permettant la gestion efficace des rendez-*

vous, en s'appuyant sur une architecture basée sur des API REST et une base de données relationnelle ?

1.3 Objectifs du projet

Les objectifs principaux de ce projet sont :

— Concevoir une application web de gestion des rendez-vous,
— Mettre en œuvre une architecture client / serveur,
— Développer des API REST avec Node.js et Express,
— Utiliser une base de données relationnelle SQLite pour la persistance des données,
— Implémenter la gestion des utilisateurs (enregistrement et connexion),
— Offrir un tableau de bord pour la planification et le suivi des rendez-vous,
— Mettre en place les opérations CRUD (Create, Read, Update, Delete),
— Tester et valider le bon fonctionnement de l'application.

1.4 Technologies utilisées

— **Frontend :** Angular
— **Backend :** Node.js, Express.js
— **Base de données :** SQLite
— **Architecture :** API REST
— **Format d'échange :** JSON
— **Outils de développement :** Visual Studio Code, Postman, Navigateur web

FIGURE 1.1 – Framework Angular pour le développement frontend

FIGURE 1.2 – Node.js pour le développement du serveur backend

FIGURE 1.3 – Framework Express.js pour la création des API REST

FIGURE 1.4 – Base de données relationnelle SQLite

1.4.1 Outils de développement

FIGURE 1.5 – Visual Studio Code – Environnement de développement

Visual Studio Code est utilisé pour le développement du frontend et du backend de l'application.

FIGURE 1.6 – Postman – Outil de test des API REST

Postman permet de tester les requêtes HTTP (GET, POST, PUT, DELETE) de l'API.

1.4.2 Technologies Web

FIGURE 1.7 – HTML – Structure des pages web

HTML est utilisé pour structurer les pages de l'application.

FIGURE 1.8 – CSS – Mise en forme de l'interface

CSS permet d'améliorer l'apparence et l'ergonomie de l'interface utilisateur.

FIGURE 1.9 – JavaScript – Logique côté client

JavaScript gère l'interactivité et la communication avec le backend.

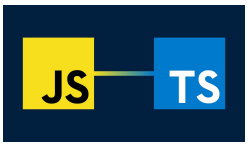

FIGURE 1.10 – TypeScript – Développement Angular

TypeScript est utilisé avec Angular pour assurer un code plus structuré et robuste.

1.4.3 Outils de rédaction du rapport

FIGURE 1.11 – LaTeX – Système de composition de documents

LaTeX est utilisé pour la rédaction de ce rapport. Il permet de produire un document structuré, professionnel et conforme aux normes académiques, avec une gestion efficace des figures, tableaux, références et du formatage global.

Chapitre 2

Analyse et Conception de la Solution

2.1 Analyse fonctionnelle

L'application permet à l'utilisateur de :

— Créer un compte et se connecter,
— Ajouter un nouveau rendez-vous,
— Consulter la liste des rendez-vous,
— Modifier un rendez-vous existant,
— Supprimer un rendez-vous,
— Visualiser les rendez-vous via un calendrier interactif.

2.2 Architecture globale de l'application

L'application repose sur une architecture en trois couches :

1. Couche présentation (Frontend)

— Développée avec Angular,
— Assure l'interface utilisateur et l'interaction,
— Consomme les API REST via des requêtes HTTP.

2. Couche logique (Backend)

— Développée avec Node.js et Express,
— Implémente les API REST,
— Traite les requêtes HTTP.

3. Couche données

— Base de données relationnelle SQLite,

— Stockage persistant des utilisateurs et rendez-vous.

Schéma de fonctionnement : Utilisateur → Angular → API REST (Express) → SQLite

Cette architecture garantit une bonne séparation des responsabilités et facilite la maintenance et l'évolution du système.

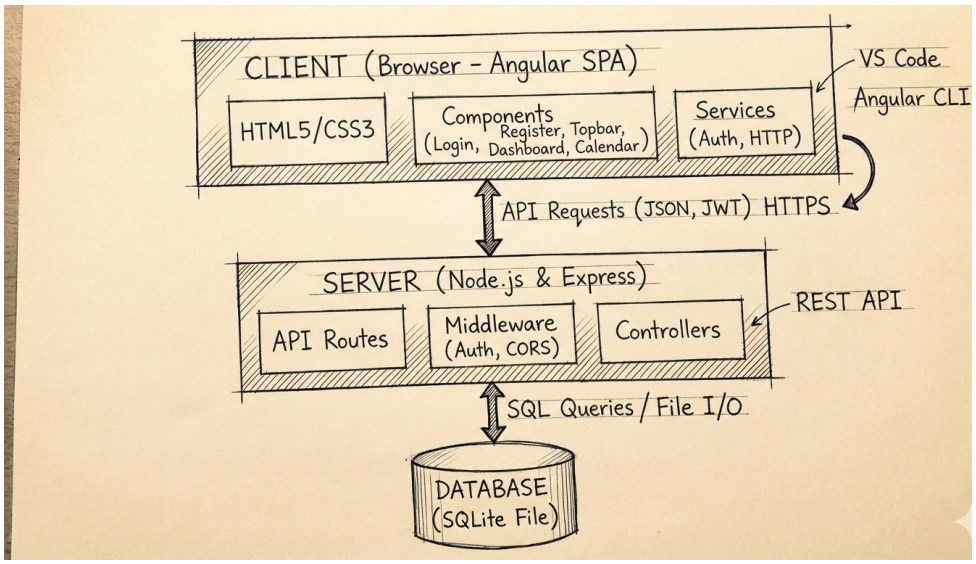

FIGURE 2.1 – Architecture du projet

2.3 Modélisation des données

Table : users

Champ	Type	Description
id	INTEGER (PK)	Identifiant utilisateur
username	TEXT	Nom d'utilisateur
email	TEXT	Adresse email
password	TEXT	Mot de passe

Table : appointments

Champ	Type	Description
id	INTEGER (PK)	Identifiant du rendez-vous
title	TEXT	Titre du rendez-vous
description	TEXT	Description
date	TEXT	Date
time	TEXT	Heure

user_id	INTEGER (FK)	Identifiant utilisateur

2.4 Conception des API REST

Méthode HTTP	**Endpoint**	**Description**
POST	/auth/register	Inscription utilisateur
POST	/auth/login	Connexion utilisateur
GET	/appointments	Récupération des rendez-vous
POST	/appointments	Ajout d'un rendez-vous
PUT	/appointments/ :id	Modification d'un rendez-vous
DELETE	/appointments/ :id	Suppression d'un rendez-vous

Ces API respectent les principes REST : séparation client/serveur, stateless, utilisation correcte des méthodes HTTP.

Chapitre 3

Réalisation et évaluation

3.1 Implémentation

Backend

- Développement avec Node.js et Express,
- Création des routes REST,
- Connexion et manipulation de la base SQLite,
- Gestion des opérations CRUD,
- Séparation claire entre routes, logique métier et accès aux données.

Frontend

- Développement avec Angular,
- Utilisation de composants et services Angular,
- Consommation des API REST via HttpClient,
- Mise en place d'un tableau de bord et d'un calendrier interactif.

3.1.1 Arborescence backend

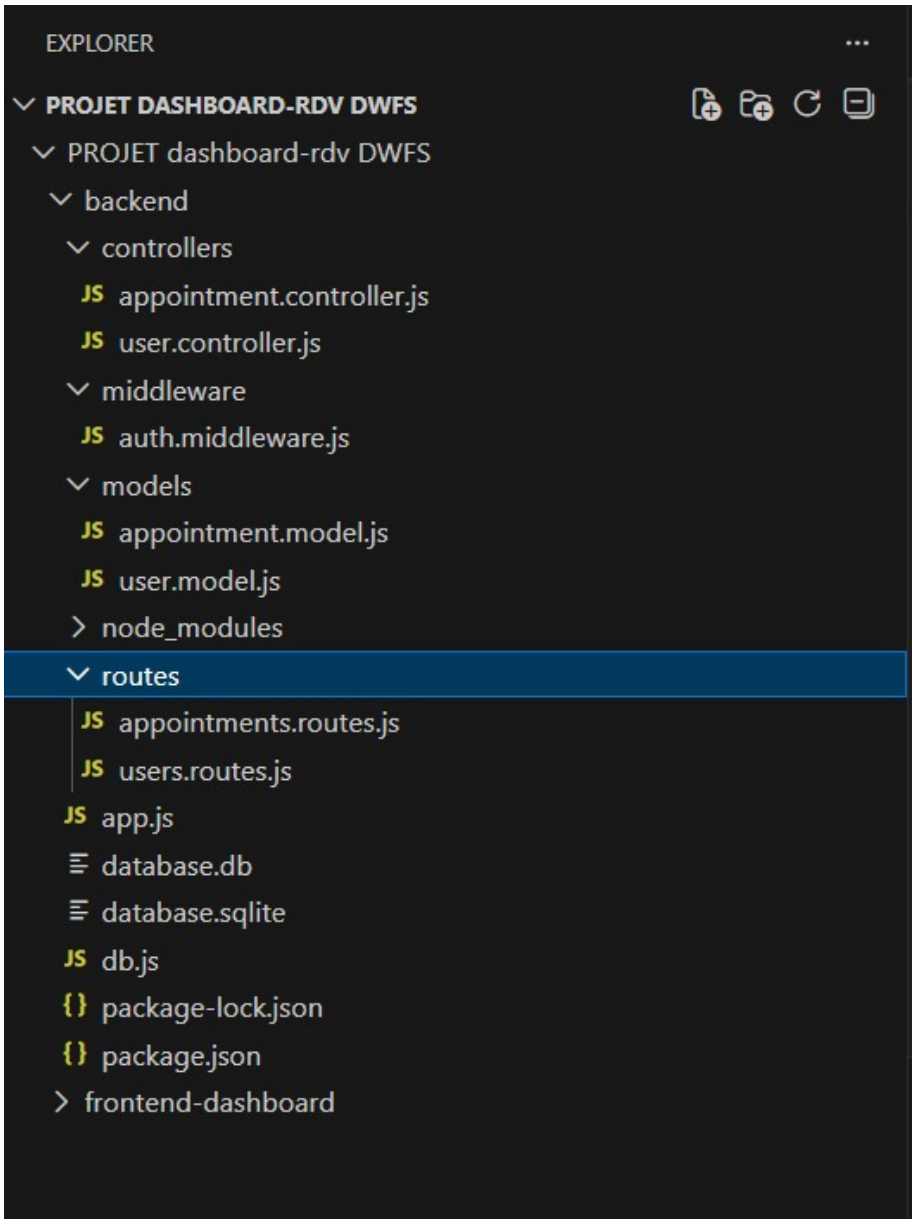

FIGURE 3.1 – Structure du projet backend

3.1.2 Arborescence frontend-dashboard

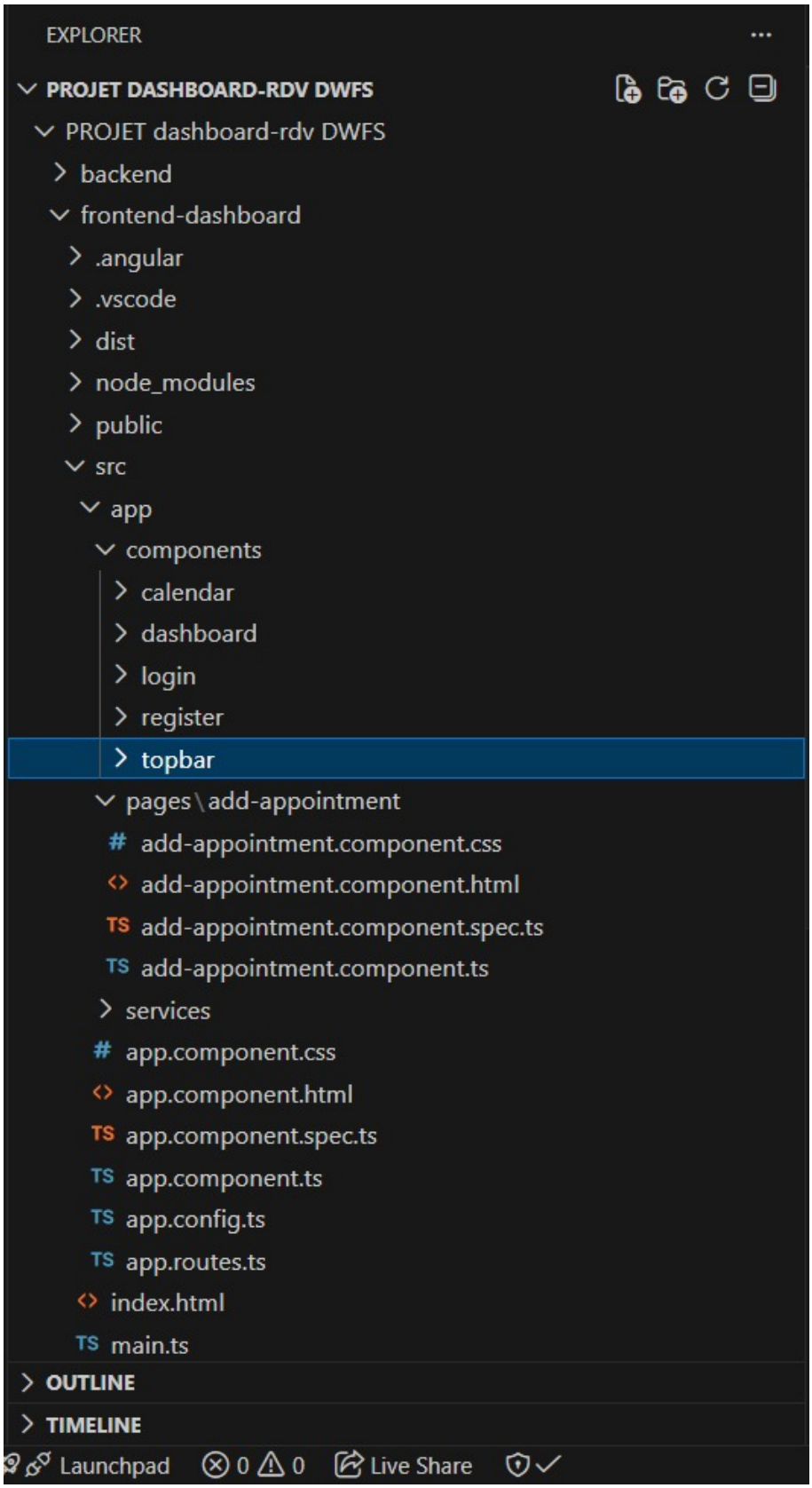

FIGURE 3.2 – Structure du projet frontend

3.2 Tests et validation

Les tests effectués comprennent :

— Tests des API REST à l'aide de Postman,

— Tests fonctionnels via l'interface utilisateur,

— Vérification de l'ajout, modification et suppression des rendez-vous,

— Tests de la gestion des utilisateurs (inscription et connexion),

— Vérification de la cohérence entre frontend, backend et base de données.

Les résultats obtenus montrent un fonctionnement correct et stable de l'application.

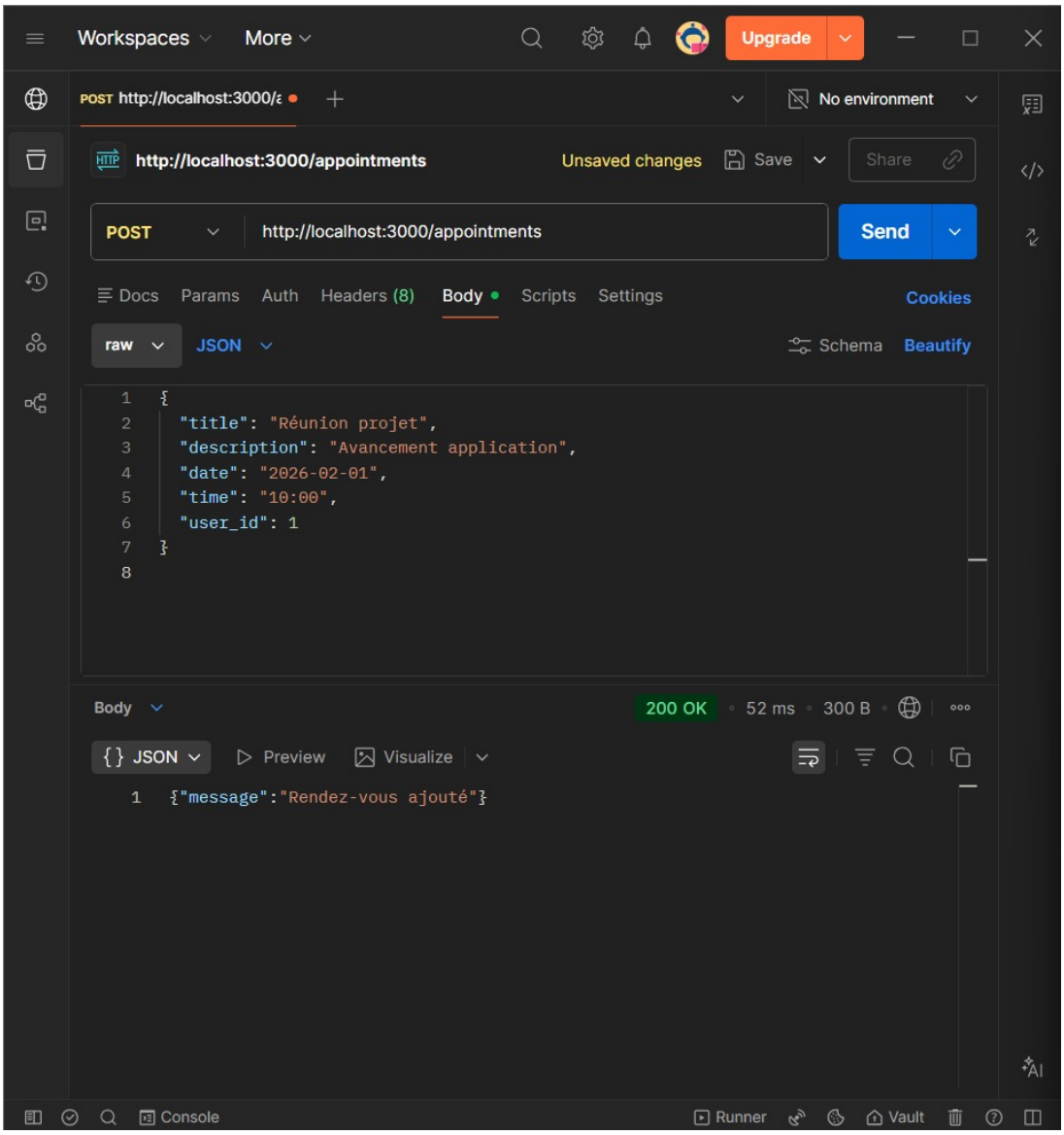

FIGURE 3.3 – Test à l'aide de Postman

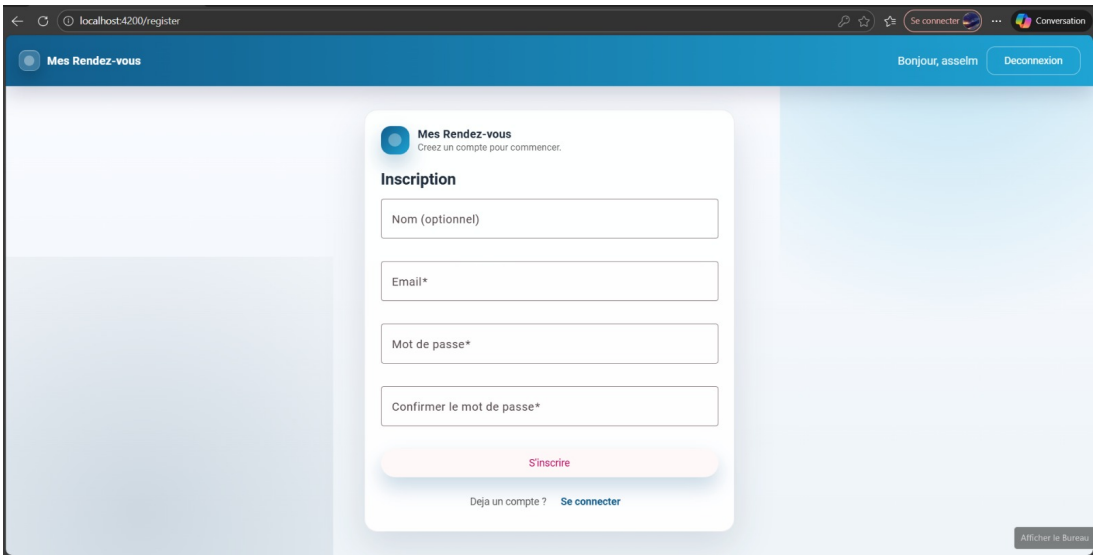

FIGURE 3.4 – Register

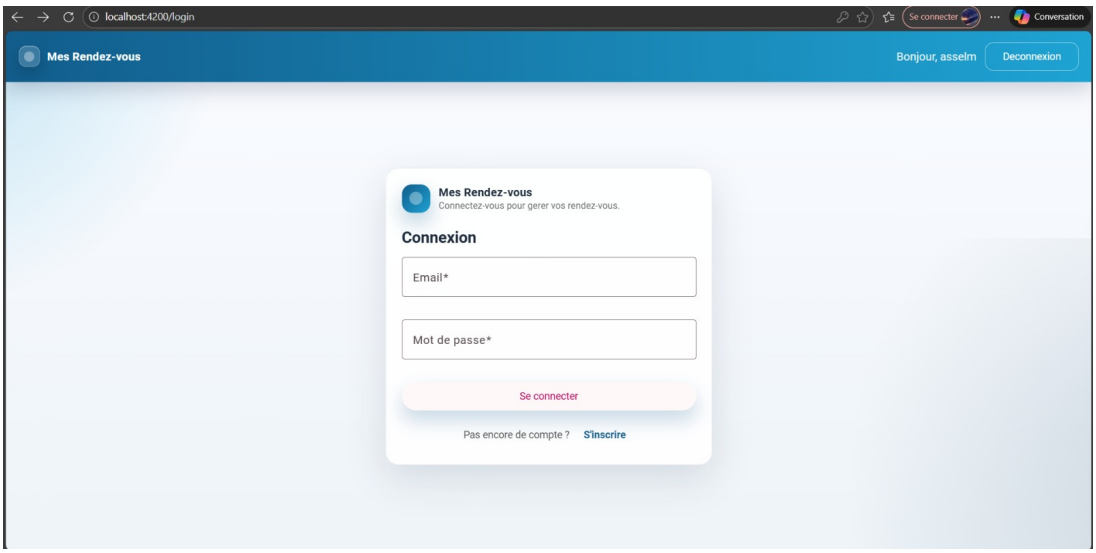

FIGURE 3.5 – Login

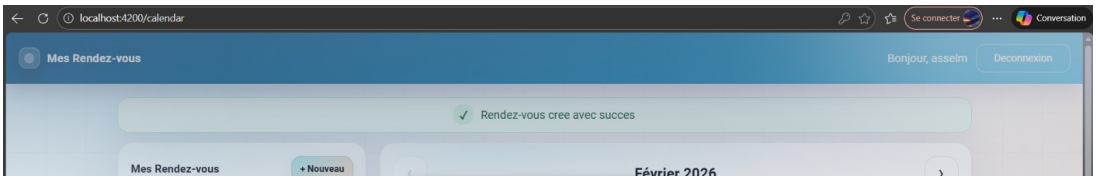

FIGURE 3.6 – Creation d'un rendez-vous

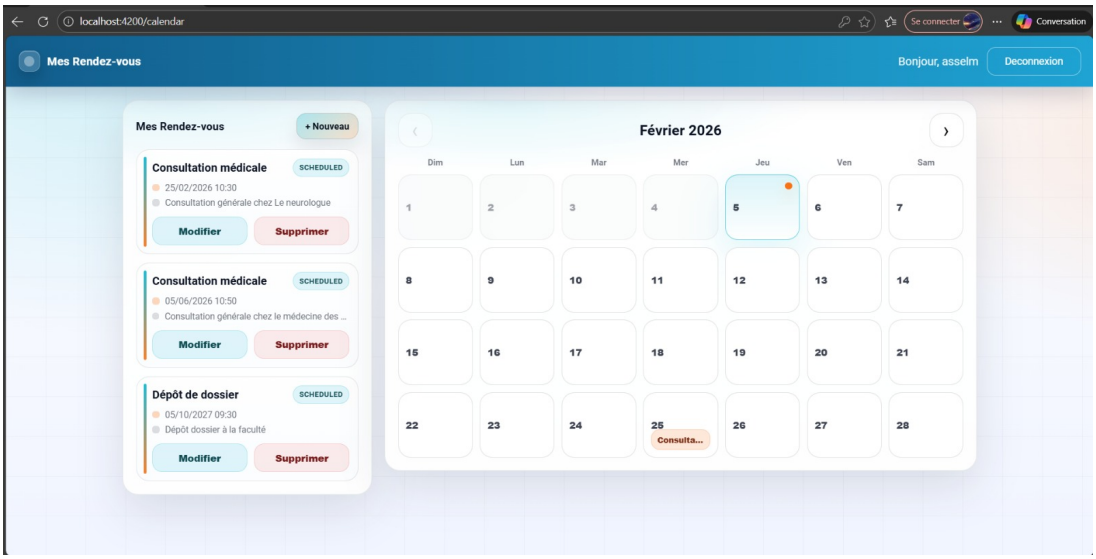

FIGURE 3.7 – Calendrier des rendez-vous

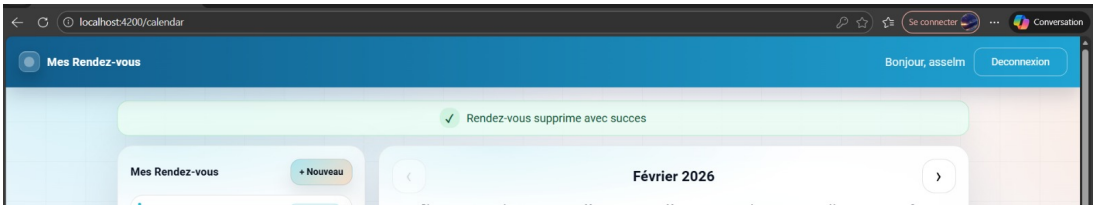

FIGURE 3.8 – Supprimer un rendez-vous

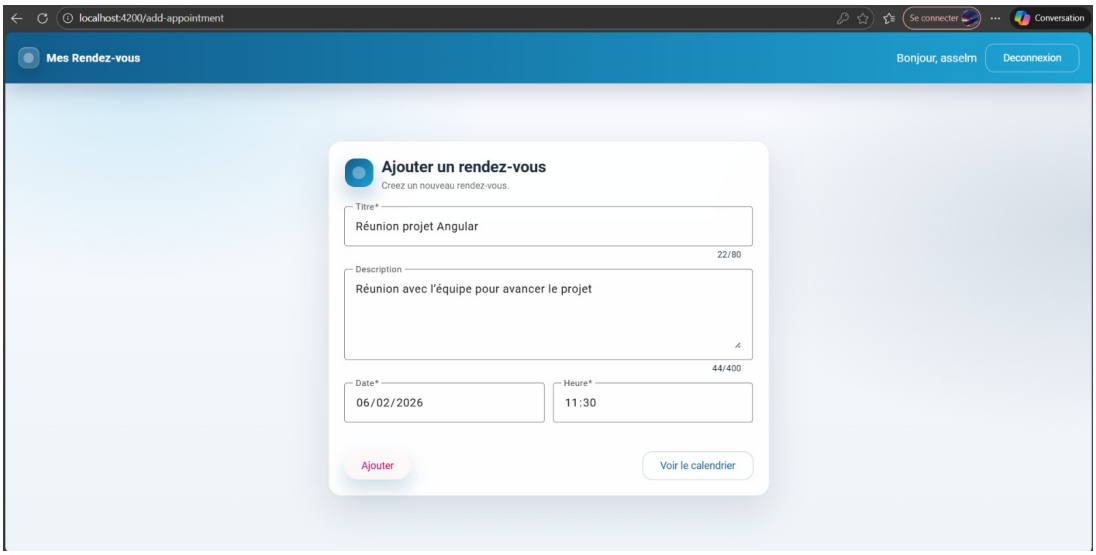

FIGURE 3.9 – Add-appointment en cas d'ajout

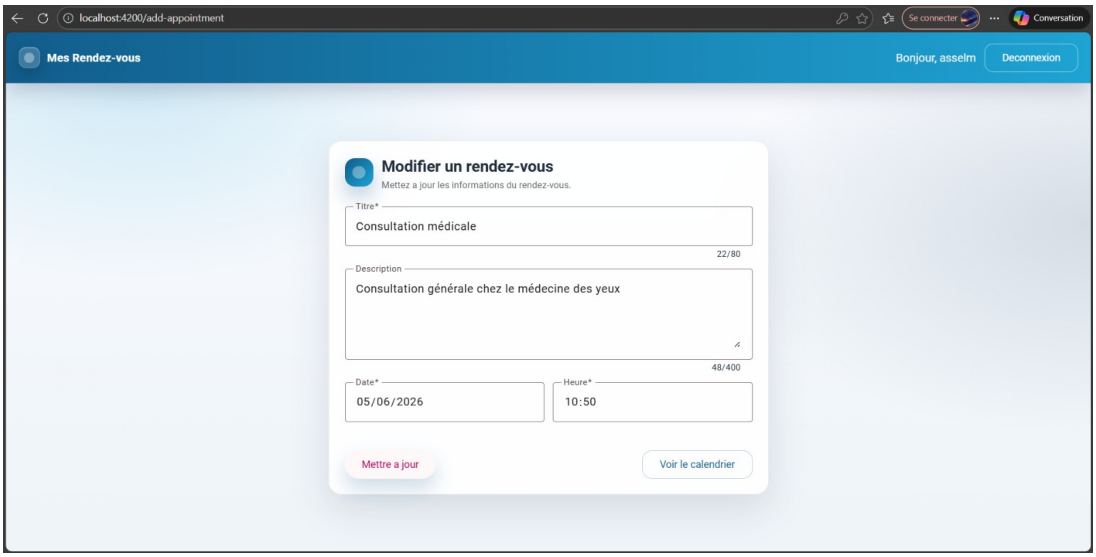

FIGURE 3.10 – Add-appointment en cas de mis a jour

3.3 Résultats obtenus

L'application permet :

— L'ajout de rendez-vous avec succès,
— L'affichage des rendez-vous enregistrés,
— La modification et la suppression des rendez-vous,
— Une communication fluide entre Angular et l'API REST,
— Un stockage fiable des données dans SQLite.

3.4 Perspectives d'amélioration

Malgré les résultats satisfaisants, certaines améliorations sont envisageables :

— Mise en place d'un système d'authentification sécurisé,
— Gestion avancée des rôles utilisateurs,
— Amélioration de l'interface graphique (UI/UX),
— Ajout de notifications automatiques,
— Déploiement sur un serveur cloud.

Chapitre 4

Conclusion Générale

Le projet **Application Web de Gestion des Rendez-vous** a constitué une excellente opportunité de mettre en pratique les connaissances théoriques acquises tout au long de la formation en Master ISI. Il a permis de se familiariser avec les concepts clés du développement web moderne et de réaliser une application complète, intégrant à la fois le backend, le frontend et la persistance des données.

Sur le plan technique, le projet a permis de :

— Concevoir et développer des **API REST** robustes avec Node.js et le framework Express, assurant une communication efficace entre le client et le serveur.

— Mettre en place une **base de données relationnelle SQLite**, permettant le stockage sécurisé et structuré des informations relatives aux utilisateurs et aux rendez-vous.

— Développer une interface utilisateur interactive et réactive avec **Angular**, incluant un tableau de bord pour la planification, la consultation et le suivi des rendez-vous.

— Implémenter toutes les opérations **CRUD** nécessaires pour gérer les rendez-vous et les utilisateurs, garantissant la fiabilité et la cohérence des données.

Le projet a également permis de se confronter à des problématiques concrètes de gestion et d'organisation d'une application web, telles que la sécurisation des informations, la gestion des conflits de rendez-vous, et la communication entre différentes couches de l'architecture.

Les tests réalisés ont confirmé la stabilité et le bon fonctionnement de l'application, tant au niveau du backend que du frontend, et ont démontré que les objectifs initiaux du projet ont été pleinement atteints.

Enfin, ce travail ouvre la voie à plusieurs perspectives d'amélioration et d'évolution, notamment :

— La mise en place d'un **système d'authentification et de gestion des rôles** plus sécurisé et performant.

— L'amélioration de l'interface utilisateur (**UI/UX**) pour rendre l'application plus intuitive et attractive.

— L'ajout de fonctionnalités avancées telles que les **notifications automatiques**, le calendrier partagé, ou encore la synchronisation avec d'autres services.
— Le **déploiement sur un serveur cloud** pour rendre l'application accessible en ligne et exploitable dans un contexte réel.

En résumé, ce projet a non seulement consolidé les compétences techniques et pratiques en développement web, mais il a également permis de développer des capacités de conception, d'analyse et de résolution de problèmes complexes, essentielles pour tout futur ingénieur en systèmes d'information.